

AI-Enhanced Analysis of Zurich-Canton Rental Listings

Which factors — size, room count, location, and amenity features — significantly influence monthly rent in Zurich canton?

Group 6

Drin Muslija

Issa Fawaz

Valdrin Dalipi

Background, problem, objective

Why we built a Swiss-rental analytics pipeline

- Background — Zurich canton has one of the most expensive rental markets in Europe; rent is the largest line in most household budgets.
- Problem — no clean, structured, canton-wide dataset exists; listing portals expose only human-readable HTML and inconsistent prices.
- Objective — build a reproducible end-to-end pipeline that scrapes, cleans, validates, enriches (LLM) and statistically tests Zurich rentals.
- Research question — which factors (size, rooms, location, features) significantly influence monthly rent in Zurich canton?
- Approach — hypothesis testing rather than black-box prediction; all inputs are auditable through a Streamlit dashboard.

What we used

Public Flatfox API + Python data stack

- Data source — flatfox.ch public JSON API; only Swiss portal not behind Cloudflare. 200 ZH apartments retained after strict filters.
- Filters — canton = ZH, full apartments only, monthly rent unit, $20 \leq m^2$ and CHF 500–15 000; excludes garages, shared flats, commercial.
- Python stack — requests + BeautifulSoup (scrape), pandas + scipy (analysis), OpenAI gpt-4o-mini (feature extraction), SQLite (persistence).
- Visualisation — matplotlib for the printed report figures, Plotly inside Streamlit for the interactive multi-tab dashboard.
- Reproducibility — pinned requirements.txt, .env-based config, deterministic content-keyed LLM cache (no duplicate API calls).

End-to-end pipeline

Eight stages, all idempotent, full chain in ~2 min on warm cache

collect_zurich	→	Flatfox JSON API → raw ZH listings + provenance fields
fetch_details	→	Threaded scrape of each detail page (BeautifulSoup)
cleaning	→	Regex normalisation, typed fields, balcony / parking flags
llm_helper	→	OpenAI feature extraction (gpt-4o-mini, SQLite-cached)
data_quality	→	Validate, dedupe, price QC report, outlier handling
database	→	SQLite persistence + canned SQL queries (NTILE, RANK, AVG)
statistics	→	7 tests — Pearson, Spearman, OLS, Welch, MWU, Shapiro
insights → UI	→	Auto business findings → Streamlit dashboard + PDF figures

Defensible numbers, not silent failures

Validation, suspicious-price audit, console QC report

- Plausibility bounds — CHF 500–15 000 / month, 15–400 m², 1–8 rooms, CHF/m² ∈ [10, 200]; non-numeric prices rejected.
- Dedup — natural key (title, price, size, rooms); also filters Tukey $k = 3$ statistical outliers on price.
- Suspicious-price audit — collector keeps raw title text and structured rent_gross; validator flags listings whose gap $\geq$ CHF 100.
- Why it matters — Flatfox occasionally serves stale public_title; without this audit, charts silently use the wrong number.
- QC console report — scraped 200 → valid 197 → suspicious 0 → missing 0 → removed 3 (statistical outliers).

collect_zurich.to_output()

src/collect_zurich.py

```
def to_output(rec: dict) -> dict:
    """Capture price provenance: raw text + parsed int +
    gross + net + charges, in one record."""
    title = rec.get("public_title") or ""
    raw_text, title_price = parse_title_price(title)
    gross = int(rec["rent_gross"])

    # Flag stale public_title (a known Flatfox artefact):
    # the visible CHF amount can lag rent_gross by re-list.
    mismatch = bool(title_price is not None
                    and abs(title_price - gross) >= 100)

    return {
        "title": title,
        "rent_price": gross,                # authoritative
        "rent_net": rec.get("rent_net"),
        "additional_costs": rec.get("rent_charges"),
        "raw_price_text": raw_text,         # what the user sees
        "title_price": title_price,        # parsed from title
        "price_mismatch": mismatch,        # audit flag
        # ... rooms, m², city, zip, listing_url ...
    }
```

Why this matters

We don't trust a single price field. We capture the structured rent_gross AND the title text AND the parsed title price — so the validator can cross-check them and we never silently use a stale number.

data_quality.validate()

src/data_quality.py

```
def validate(df) -> tuple[pd.DataFrame, QualityReport]:
    """Drop dupes / impossible / outliers; flag suspicious."""
    report = QualityReport(n_input=len(df))

    # ... drop dupes, missing prices, impossible bounds,
    #     IQR k=3 statistical outliers (omitted for slide) ...

    # Suspicious-price audit: title CHF vs structured price
    if "title_price" in df.columns:
        tp = pd.to_numeric(df["title_price"], errors="coerce")
        gap = (tp - df["price"]).abs()
        suspicious = (gap >= PRICE_MISMATCH_TOLERANCE)
        suspicious = suspicious.fillna(False)
        if suspicious.any():
            report.n_suspicious_price = int(suspicious.sum())
            report.suspicious_examples = (
                df.loc[suspicious, AUDIT_COLS].reset_index(drop=True)
            )

    return df, report
```

Why this matters

Validator never silently keeps a mismatch.

Every flagged listing lands in the QC report with its Flatfox URL — a human can audit each one in one click.

llm_helper.LLMProcessor.extract_features()

src/llm_helper.py

```
def extract_features(self, description: str) -> dict:
    """Return {balcony, parking, furnished}.
    Content-keyed SQLite cache → never pay twice."""
    if not description:
        return {k: 0 for k in self.SCHEMA_KEYS}

    cache_key = self._cache_key(description)    # hash(prompt_v|model|desc)
    cached = self._cache_get(cache_key)
    if cached is not None:
        self.calls_cache += 1
        return cached                          # cache hit → no API call

    if self._client is not None:
        try:
            result = self._extract_via_openai(description)
            self._cache_put(cache_key, result)
            self.calls_openai += 1
            return result
        except Exception as exc:
            ... # rate-limit retry, 401 disables LLM for session

    self.calls_fallback += 1
    return self._extract_via_regex(description) # always-available fallback
```

Why this matters

Three-tier strategy: cache hit →
OpenAI → regex fallback.

Cache is keyed by prompt version,
so bumping PROMPT_VERSION
invalidates everything without
per-row tracking.

statistics.correlation_price_size()

src/statistics.py

```
def correlation_price_size(df: pd.DataFrame) -> TestResult:
    """Pearson r — living space vs monthly rent."""
    sub = df[["size", "price"]].dropna()
    r, p = pearsonr(sub["size"], sub["price"])
    r2 = r ** 2
    return TestResult(
        name="pearson_size_price",
        statistic=float(r),
        p_value=float(p),
        effect_size=float(r2),
        effect_size_label=(
            f"R² = {r2:.3f} — size explains "
            f"{r2*100:.1f}% of price variance"
        ),
        interpretation=(
            f"r = {r:.3f} ({_correlation_strength(r)} "
            f"positive relationship). {_p_phrase(p)}. "
            f"Each extra m² is associated with higher rent."
        ),
        n=len(sub),
    )
```

Why this matters

Every test ships an effect size
AND a plain-language sentence.

The dashboard and the insights
module render those strings
verbatim — statistical literacy is
not assumed.

pipeline.run_pipeline()

src/pipeline.py

```
def run_pipeline(use_llm=True, max_listings=200) -> dict:
    """Zurich-canton end-to-end pipeline."""
    base = load_or_collect_zurich(target=max_listings)
    enriched = fetch_details(base)           # threaded scrape
    rows = [_to_cleaner_shape(r) for r in enriched]
    cleaned = clean_records(rows)           # regex normalisation

    if use_llm:
        cleaned = enrich_records(cleaned, LLMProcessor()) # OpenAI + cache

    df = records_to_dataframe(cleaned)
    df["chf_per_m2"] = (df["price"] / df["size"]).round(2)

    df_clean, quality_report = validate(df)    # QC + suspicious audit
    quality_report.print_summary()             # console QC report

    with HousingDatabase(DB_PATH) as db:
        db.save(df_clean)                    # SQLite persistence

    return {
        "df": df_clean,
        "tests": run_all_tests(df_clean),
        "insights": generate_insights(df_clean),
        "quality_report": quality_report.as_dict(),
    }
```

Why this matters

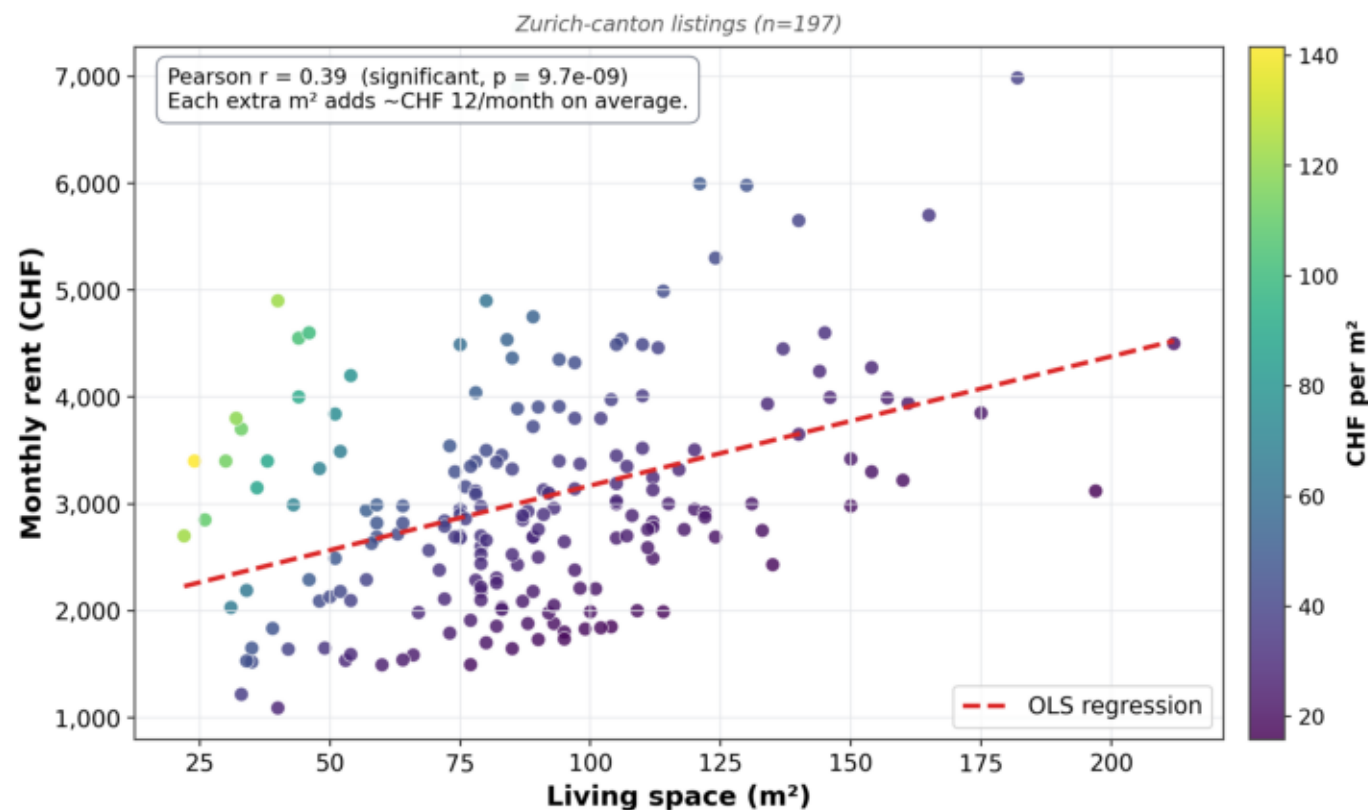
One function, eight stages,
deterministic on a warm cache.

Same entry-point is called by the
CLI, the Streamlit app, and the
Jupyter notebook — no duplicated
orchestration.

Larger flats usually have higher monthly rent

Pearson $r = 0.39$ · slope $\approx$ CHF 12 / m² / month · $n = 197$

Larger flats usually have higher monthly rent



Take-away

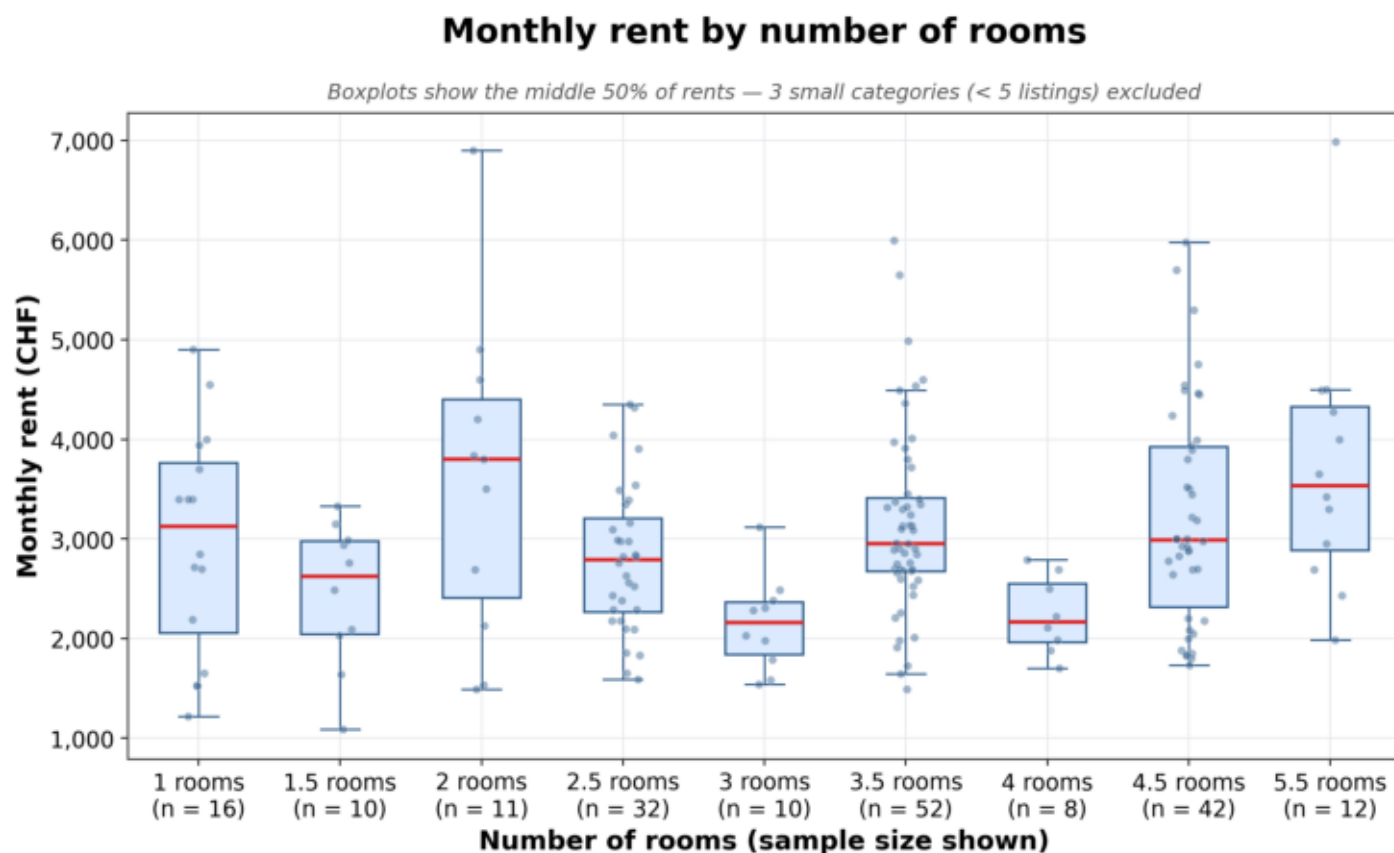
Size is the single strongest linear predictor of monthly rent.

But spread is large: same-size flats can differ by CHF 2 000+ depending on location and amenities.

DEMO

Monthly rent by number of rooms

Boxplot of price per room category (≥ 5 listings each)



Take-away

Room count is a much weaker price signal than m^2 (Spearman $\rho \approx 0.15$).

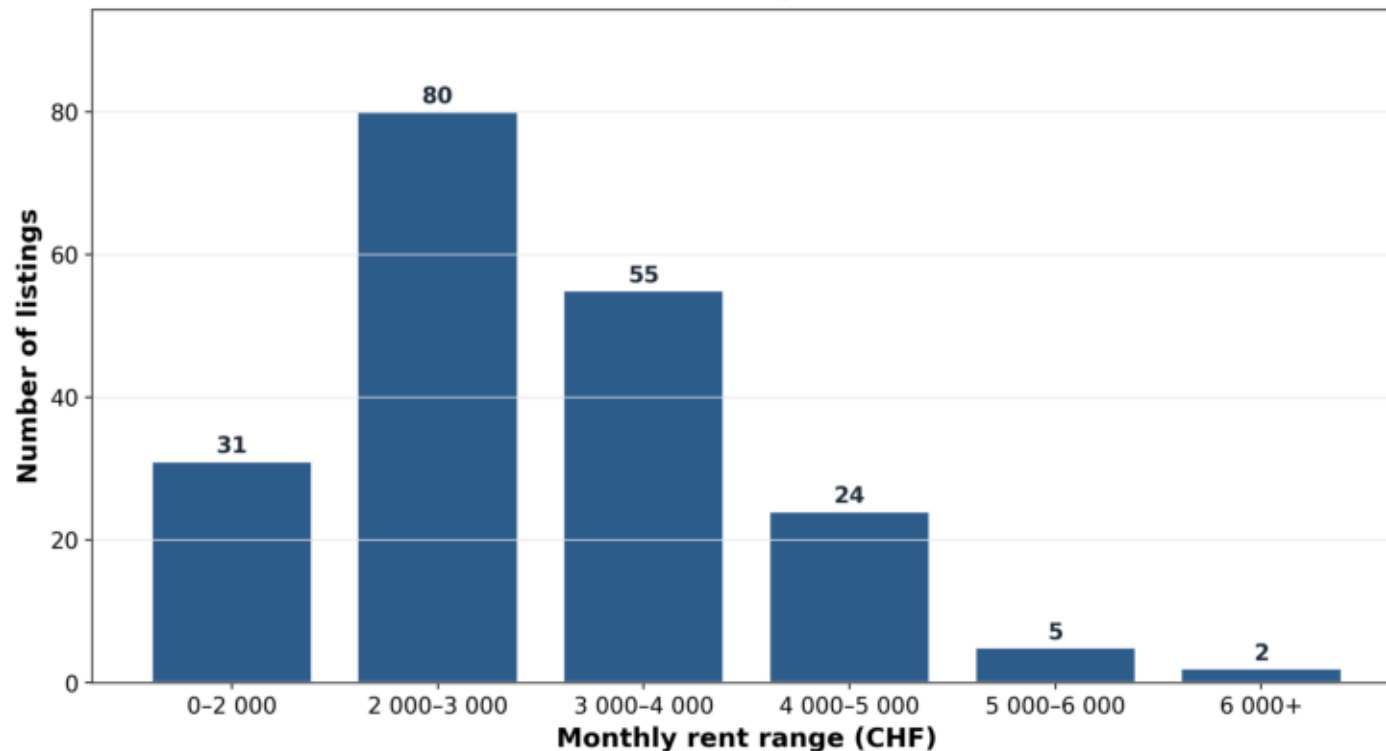
Same-room-count flats vary widely in m^2 , which dilutes the signal.

How many listings fall into each rent range?

Fixed bins: 0–2 000, 2 000–3 000, ... 6 000+

How many listings fall into each rent range?

Zurich-canton listings (n = 197)



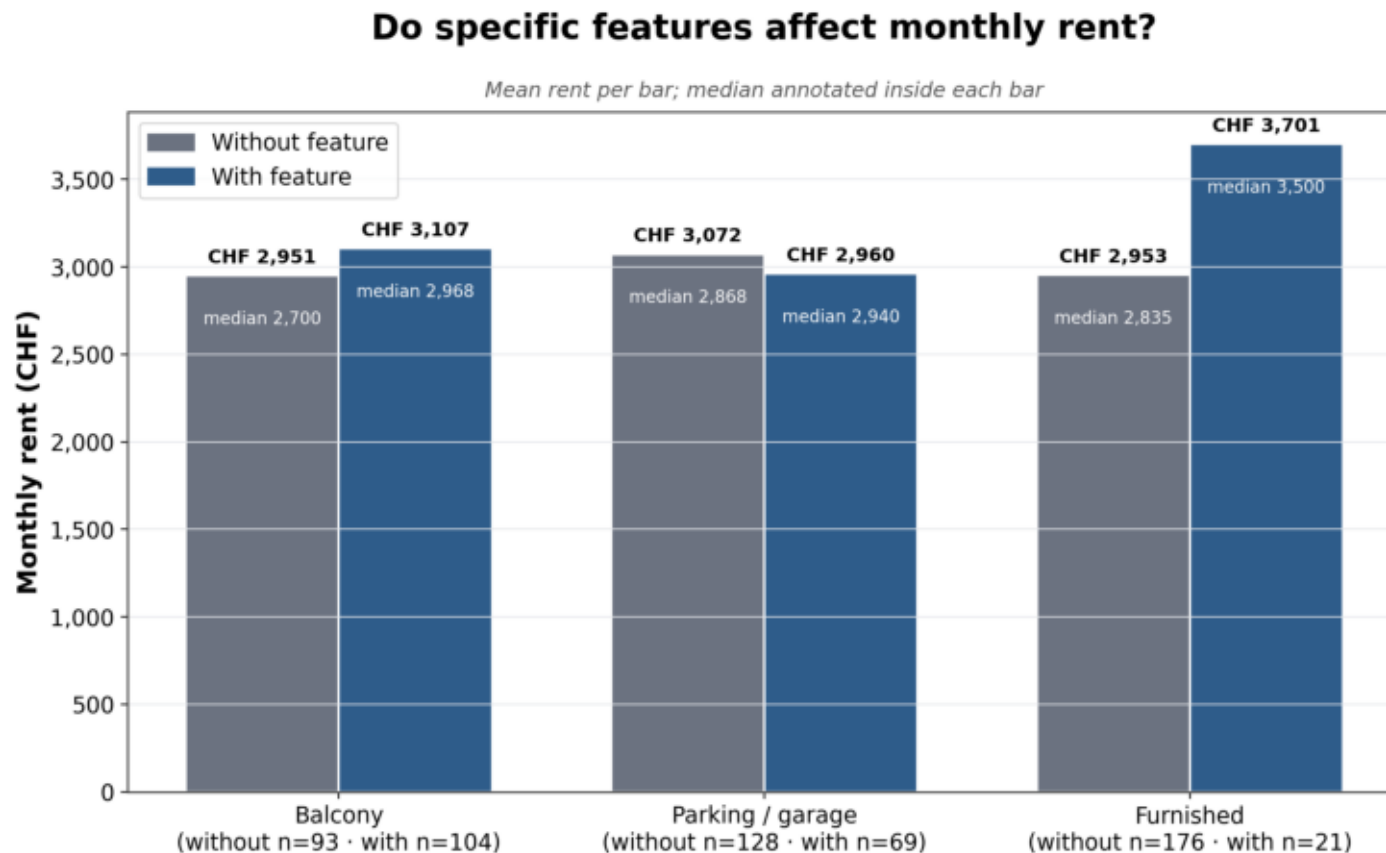
Take-away

Modal bucket is CHF 2 000–3 000 (80 listings).

~70% of Zurich-canton listings rent between CHF 2 000 and 4 000 / month. Only 7 cross the CHF 6 000 line.

Do specific features affect monthly rent?

Mean + median rent, with sample sizes



Take-away

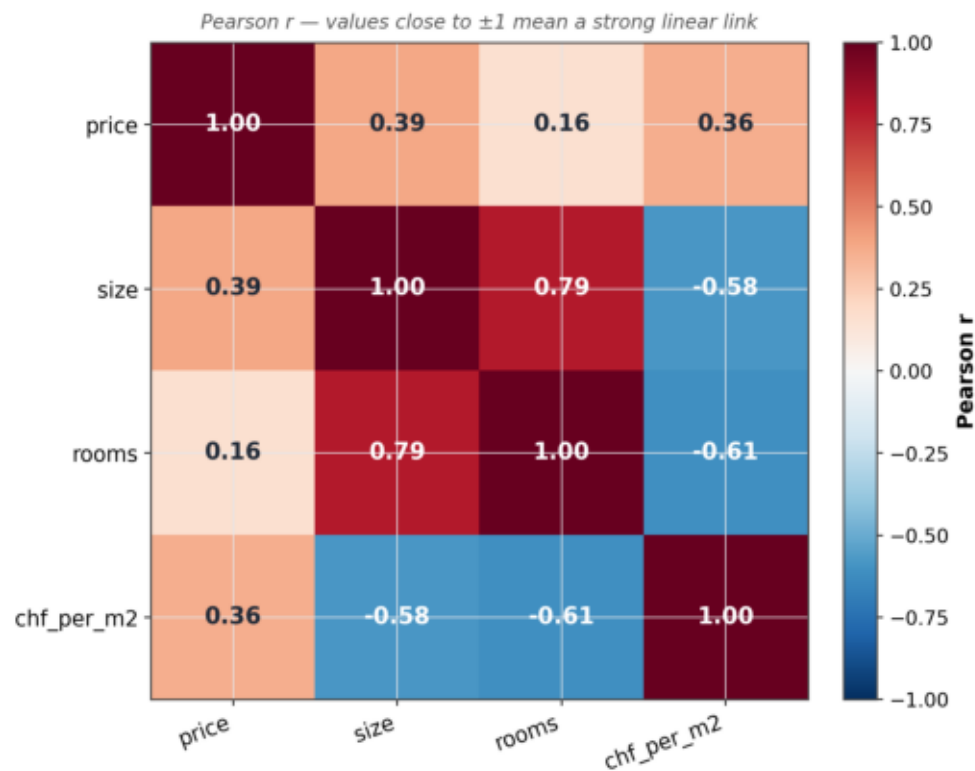
Furnished is the only feature with a large effect (+CHF 750, ~25%).

Balcony +CHF 150 not significant.
Parking shows a small negative gap, driven by location confounding.

Correlation between numeric housing features

Pearson r — values close to ± 1 mean a strong linear link

Correlation between numeric housing features



Take-away

Strongest correlations:

- size $\leftrightarrow$ price: +0.39
- size $\leftrightarrow$ rooms: +0.79
- size $\leftrightarrow$ CHF/m²: -0.58

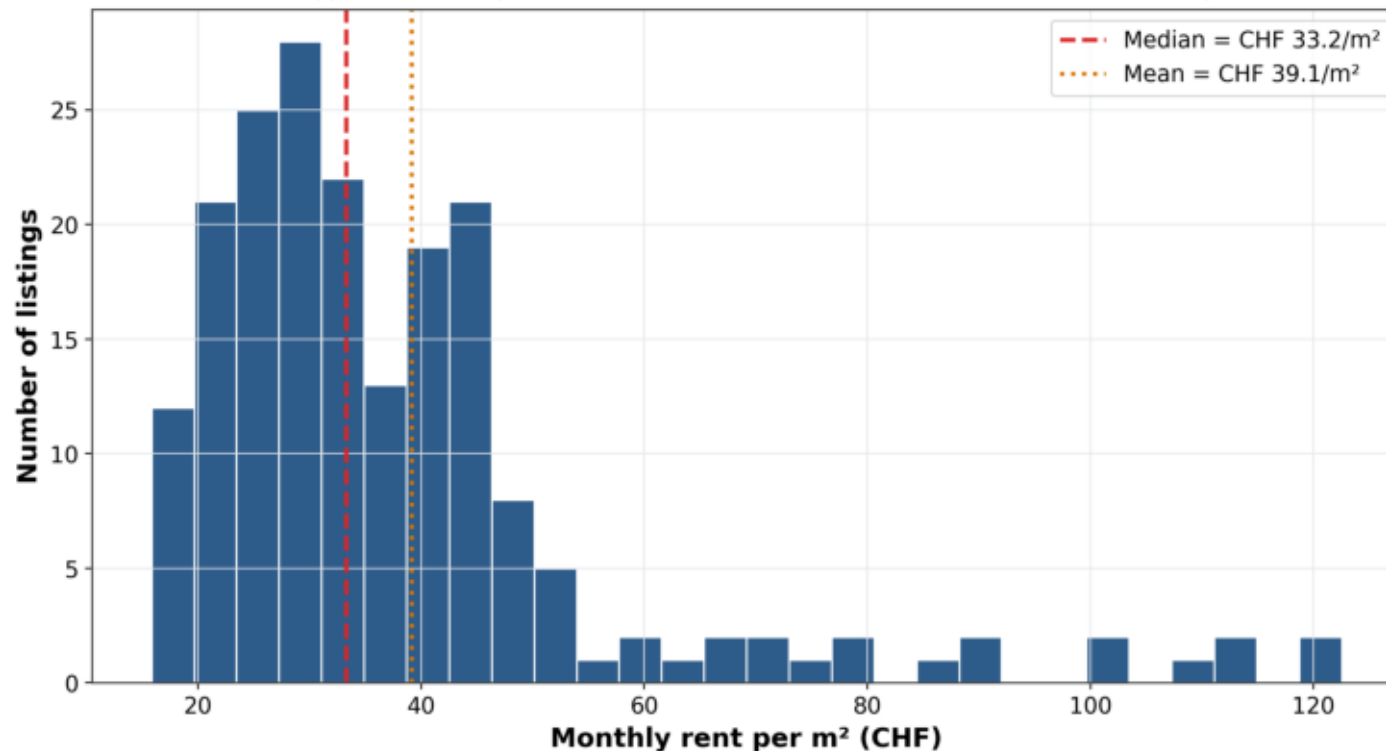
Small flats charge a per-m² premium.

Distribution of monthly rent per square metre

Median = CHF 33.2/m² · Mean = CHF 39.1/m² · right-skewed

Distribution of monthly rent per square metre

Plot capped at the 99th percentile (2 outliers excluded from view; statistics use the full sample)



Take-away

Median (33.2) < Mean (39.1) → right-skewed distribution.

A small tail of luxury listings pulls the mean upward; the typical listing sits around CHF 33/m².

What the data tells us — and what it doesn't

Interpretation + honest limitations

- Findings — living space is the single strongest linear predictor of rent; ~CHF 12/m² per month. Rooms is a noisy proxy by comparison.
- Amenities — furnished is the only feature with a clear signal (+CHF 750 mean, ~25%); balcony and parking effects are within noise.
- Small-flat premium — size ↔ CHF/m² correlation is -0.58 ; micro-units consistently extract a price-per-area markup.
- Limitation — Flatfox-only sample of 200 listings; Homegate, ImmoScout and newhome sit behind Cloudflare.
- Limitation — snapshot in time; rents move with season and mortgage rates. A longitudinal study would be a natural follow-up.
- Limitation — LLM feature extraction is fast/cheap but imperfect; validated against a regex fallback on unambiguous cases.

What we delivered

Reproducible pipeline + defensible answers

- Reproducible end-to-end pipeline turns Flatfox listings into a validated, statistically-tested dataset with full audit trail.
- Research question answered — living space dominates rent; room count is a noisy proxy; only furnishing carries a clear premium.
- Engineering contribution — price-QC report cross-checks scraped prices against the listing's own title text and flags mismatches.
- Outputs — Streamlit dashboard (interactive), six light-theme PDF figures (printed report), SQLite DB (downstream queries).
- Future work — multi-canton expansion, time-series snapshots, LLM short-circuit when regex is already confident, external centroid CSV.

Thank you

Questions?

Group 6 · Scientific Programming — FS 2026

Drin Muslija · Issa Fawaz · Valdrin Dalipi

Source code & figures: github.com/Drin06577/scientific_programming_LN

Where everything lives

For graders who want to inspect the code

- GitHub repo — github.com/Drin06577/scientific_programming_LN
- Pipeline entry-point — ``python -m src.pipeline`` (prints QC report)
- Dashboard — ``streamlit run app/streamlit_app.py``
- Report figures — ``python -m src.report_figures`` → `reports/figures/`
- Validation rules — `src/data_quality.py` (suspicious-price audit)
- Project context — `CLAUDE.md`, `HOW_TO_RUN.md`, `README.md`